

RFTag Messaging SOP

Sending and Receiving LoRa Messages from a Raspberry Pi

Firmware 2.2.2-rel (b5b3ef9b4470)	Host Raspberry Pi OS Lite 64-bit (Debian 13)	Verified 2026-08-19
---	--	-------------------------------

Scope. Putting an RFTag board into a group and exchanging LoRa text messages using a Raspberry Pi as the host.

Every terminal block in this document is real output, captured from a Raspberry Pi 3B running Raspberry Pi OS Lite (64-bit, Debian 13 "trixie") against RFTag firmware **2.2.2-rel** (build **b5b3ef9b4470**) on 2026-08-19. Nothing is illustrative or reconstructed.

1. Prerequisites

Hardware

Item	Requirement
RFTag board	Powered and running application firmware. Enumerates as USB 1915:520f .
USB cable	Must carry data. A charge-only cable is the most common failure — the board powers up and looks alive but never appears to the Pi.
Host	Raspberry Pi (or any Linux box / Mac). On a Pi 3B all four USB ports share a ~1.2 A budget.
Second board	Required only to confirm messages are actually received over the air.

Software

Tool	Version used	On a fresh Pi OS Lite	If missing
bash	5.2.37	Always — Debian Priority: required	n/a
python3	3.13.5	Yes, but Priority: optional	<code>sudo apt install python3</code>
pyserial	3.5	No — must be installed	<code>./setup.sh</code>
git	2.47.3	Yes, but Priority: optional	<code>sudo apt install git</code>
dialout group	—	Yes, the image's first user is a member	<code>sudo usermod -a -G dialout \$USER</code> , then re-login

Verify the environment:

```
pi@raspberrypi:~/script_pi $ bash --version | head -1
GNU bash, version 5.2.37(1)-release (aarch64-unknown-linux-gnu)
pi@raspberrypi:~/script_pi $ python3 -V
Python 3.13.5
pi@raspberrypi:~/script_pi $ git --version
git version 2.47.3
pi@raspberrypi:~/script_pi $ python3 -c "import serial; print('pyserial', serial.__version__)"
pyserial 3.5
pi@raspberrypi:~/script_pi $ id -nG | tr ' ' '\n' | grep -qx dialout && echo "dialout: ok"
dialout: ok
```

2. Procedure

Step 1 — Clone the repository

```
pi@raspberrypi:~ $ git clone https://github.com/Everbliss-Green/script_pi.git
Cloning into 'script_pi'...
pi@raspberrypi:~ $ cd script_pi
```

Step 2 — Run setup (once per machine)

```
pi@raspberrypi:~/script_pi $ ./setup.sh
==> Installing pyserial
Reading package lists...
Building dependency tree...
Reading state information...
python3-serial is already the newest version (3.5-2).
0 upgraded, 0 newly installed, 0 to remove and 95 not upgraded.
==> Checking serial port access
    'pi' is already in the dialout group.

==> Done. Plug the board in and try:
    ./rftag_cli.py ports
```

On a machine where pyserial is genuinely absent this installs it. If it reports **adding** you to the **dialout** group, log out and back in before continuing — group membership is only applied at login.

Step 3 — Confirm the board is detected

```
pi@raspberrypi:~/script_pi $ ./rftag_cli.py ports
Found 3 RFTag interface(s):
/dev/ttyACM0      RFTag  serial=24BF12C62A33FB33
/dev/ttyACM1      RFTag  serial=24BF12C62A33FB33
/dev/ttyACM2      RFTag  serial=24BF12C62A33FB33

Shell interface: /dev/ttyACM0
```

Three interfaces is correct: shell, log console, and mcumgr. The scripts probe for the one that answers with a **uart:~\$** prompt, so the device number does not have to be known in advance.

If this reports no board found, stop here and work through **Section 5 — Troubleshooting** before continuing. The most common cause is a charge-only USB cable.

Step 4 — Check the board's identity and radio (optional)

```
pi@raspberrypi:~/script_pi $ ./rftag_cli.py info
Firmware 2.2.2-rel
Build b5b3ef9b4470
MAC Device MAC: D4:B5:2C:61:A9:E0
Battery Battery SOC: 99.99%
Group ID Group ID: 0x01E38D46 (31690054)
Username Username:
Status Current status flags: 0x0201
Active: leader

Radio:
Active Radio Config:
Frequency: 922500000 Hz (922.500 MHz)
Bandwidth: 500 kHz
Spreading: SF10
Coding Rate: 4/5
Preamble: 16 symbols
TX Power: 22 dBm
Max pkt airtime: 211 ms
```

Note the **MAC** — it identifies this board in messages received by peers.

Step 5 — Set the group and radio profile (once per board)

This clears device data. The firmware clears stored members, messages and location history on any group-ID change. The script sets the group ID first so the seven settings applied afterwards survive that wipe.

```
pi@raspberrypi:~/script_pi $ ./set_group.sh
Applying profile 'mountaineering' -- Taiwan / AS923 channel 41, long range, 2 min beacon
Note: changing the group ID clears stored members, messages and location history.

[set ] rftag settings groupid set 31690054
      Group ID set to: 0x01E38D46 (31690054)
[set ] rftag settings lora freq 922500000
      Frequency set to: 922500000 Hz (922.500 MHz)
[set ] rftag settings lora bw 500
      Bandwidth set to: 500 kHz
[set ] rftag settings lora sf 10
      Spreading factor set to: SF10
[set ] rftag settings lora cr 5
      Coding rate set to: 4/5
[set ] rftag settings lora preamble 16
      Preamble set to: 16 symbols
[set ] rftag settings lora power 22
      TX power set to: 22 dBm
[set ] rftag settings timing interval 120
      location_update_interval set to: 120 sec

Verifying (reading values back from the device):
[ok ] group id      Group ID: 0x01E38D46 (31690054)
[ok ] frequency     Frequency: 922500000 Hz (922.500 MHz)
[ok ] bandwidth     Bandwidth: 500 kHz
[ok ] spreading factor Spreading: SF10
[ok ] coding rate    Coding Rate: 4/5
[ok ] preamble       Preamble: 16 symbols
[ok ] tx power       TX Power: 22 dBm
[ok ] location interval location_update_interval: 120 sec (range: 1-3600)

Profile 'mountaineering' applied and verified (8/8 values read back correctly).
```

Exit code 0. **Every value is read back from the device and compared** — the script does not trust the device's "set" acknowledgement alone, because a value can be reported as set without being persisted.

Run this on **every board that must talk to each other**. Devices that disagree on any radio parameter will not hear one another.

Preview without changing anything:

```
pi@raspberrypi:~/script_pi $ ./set_group.sh --dry-run
Would apply profile 'mountaineering':
  rftag settings groupid set 31690054
    then verify with: rftag settings groupid get  ~=  \{31690054\}
  rftag settings lora freq 922500000
    then verify with: rftag settings lora get  ~=  Frequency:\s+922500000 Hz
  rftag settings lora bw 500
```

Step 6 — Send a message

```
pi@raspberrypi:~/script_pi $ ./send_message.sh hello from the SOP
Sending: hello from the SOP
Text Message (36 bytes):
  Text: 'hello from the SOP'
  MAC: D4:B5:2C:61:A9:E0
  Timestamp: 1787104784
  Group ID: 0x01E38D46
  Status: 0x0201
Hex:
D4 B5 2C 61 A9 E0 10 0E 85 6A 68 3F 5D D4 CC BF
0F D9 A7 9D 34 A5 9A A0 97 56 B4 F1 AC 95 EB 55
87 4F 4D 13
Base64:
1LUsYangEA6Famg/XdTMvw/Zp500pZqgl1a08ayV61WHT00T
[QUEUED] Message stored for LoRa transmission
```

[QUEUED] means the message was serialized, encrypted and handed to the radio. Run the script as many times as required — one message per call.

All three argument forms work; quoting is optional:

```
./send_message.sh hello
./send_message.sh "hello from the trail"
./send_message.sh hello from the trail
```

Called with no argument it refuses rather than sending an empty message:

```
pi@raspberrypi:~/script_pi $ ./send_message.sh
Usage: send_message.sh <message>

Examples:
  send_message.sh hello
  send_message.sh "hello from the trail"

The board is found automatically over USB. Run ./setup.sh first if this is a
fresh machine.
```

Exit code **1**.

3. Receiving messages

Reading a message deletes it from the board. The firmware pops each message off the queue and there is no peek operation. Everything read is printed once and is then gone. Use **--count** if you only need the depth.

Check the queue without consuming it

```
pi@raspberrypi:~/script_pi $ ./receive_messages.sh --count
Incoming messages waiting: 1
```

Read everything waiting, then exit

```
pi@raspberrypi:~/script_pi $ ./receive_messages.sh
4 message(s) waiting:

[1] from D1:D7:E6:13:AD:1A  group text    2026-08-19 09:53:17
    status 0x5201 [leader, unknown bits 0x5200]
    "H" (1 bytes)
[2] from D1:D7:E6:13:AD:1A  group text    2026-08-19 09:53:27
    status 0x5201 [leader, unknown bits 0x5200]
    "Tuityiiy575785" (14 bytes)
[3] from D1:D7:E6:13:AD:1A  group text    2026-08-19 09:53:37
    status 0x5201 [leader, unknown bits 0x5200]
    "$??&&(&(&)" (10 bytes)
[4] from D1:D7:E6:13:AD:1A  group text    2026-08-19 09:53:48
    status 0x5201 [leader, unknown bits 0x5200]
    "$??&&(&(/$(/" (12 bytes)
```

Listen continuously

`--watch` never exits on its own. It drains anything already queued, then polls and prints each message as it arrives. Ctrl-C stops it.

```
pi@raspberrypi:~/script_pi $ ./receive_messages.sh --watch
Watching for messages on /dev/ttyACM0 -- polling every 2.0s. Ctrl-C to stop.

Waiting...

--- 09:56:46 ---
[1] from D1:D7:E6:13:AD:1A  group text    2026-08-19 09:56:45
    status 0x5201 [leader, unknown bits 0x5200]
    "Ydyddy" (6 bytes)

--- 09:56:58 ---
[2] from D1:D7:E6:13:AD:1A  group text    2026-08-19 09:56:56
    status 0x5201 [leader, unknown bits 0x5200]
    "Hguguguugguugguiggi" (19 bytes)

--- 09:57:05 ---
[3] from D1:D7:E6:13:AD:1A  direct text    2026-08-19 09:57:04
    status 0x5201 [leader, unknown bits 0x5200]
    "Hvvuvh" (6 bytes)

--- 09:57:12 ---
[4] from D1:D7:E6:13:AD:1A  group text    2026-08-19 09:57:09
    status 0x5201 [leader, unknown bits 0x5200]
    "Hvguug" (6 bytes)
^C
Stopped. 4 message(s) received.
```

The `--- HH:MM:SS ---` header marks each poll that found something. Messages arriving mid-session are printed within one poll interval (default 2 s; change with `--interval`).

Reading the output

Field	Meaning
from D1:D7:E6:13:AD:1A	The sending board's MAC
group text / direct text	Message type. Also decoded: join, location, delivery receipt, targeted resend
2026-08-19 09:56:45	Timestamp from the sender , not the receiver
status 0x5201 [leader, unknown bits 0x5200]	Sender's status flags — see below
"Ydyddy" (6 bytes)	Payload. Binary payloads (e.g. delivery receipts) print as hex

About unknown bits. The flag table is read from the device itself (`rftag settings status list`) so it can never drift from firmware, but that table only names bits 0–8. A status like `0x5201` contains bits outside it. Those are reported rather than silently dropped — otherwise `0x5201` would be displayed as plain `leader`, which is wrong.

Other options

```
./receive_messages.sh --watch --interval 5 # slower polling
./receive_messages.sh --clear             # discard everything unread
./receive_messages.sh --help
```

4. Verifying a two-board exchange

1. Run `./set_group.sh` on **both** boards.
2. On the receiving Pi: `./receive_messages.sh --watch`
3. From the other board, send a message.
4. It appears in the watch output within one poll interval, with the **sending board's MAC** in the `from` field.

If nothing arrives, confirm both boards report the same group ID and identical radio settings via `./rftag_cli.py info`.

5. Troubleshooting

No board found

```
error: No RFTag board found (looking for USB 1915:520f).
- Is the board plugged in and powered?
- Is the USB cable a data cable? Charge-only cables are the
  usual culprit: the board looks alive but never enumerates.
```

In order of likelihood:

1. **Charge-only USB cable.** By far the most common cause. Confirm with `lsusb` — you are looking for `1915:520f`.
2. **Board not powered or not running the application firmware.**
3. **USB current.** On a Pi 3B all ports share ~1.2 A. If the board enumerates and then disappears, that is the brownout signature — remove other peripherals or use a powered hub.

device reports readiness to read but returned no data

```
error: serial failure: device reports readiness to read but returned no data (device disconnected or multiple access on port?)
```

Two programs are using the serial port at once. Only one may hold `/dev/ttyACM0`. This happens when a `--watch` session is already running in another terminal, or a stray process was left behind. It is not a hardware fault, but the two processes do steal each other's bytes, so data read during the collision may be corrupted.

Find and stop the other holder:

```
ps aux | grep rftag_cli | grep -v grep
sudo fuser -v /dev/ttyACM0
```

Permission denied on the port

You are not in the `dialout` group, or have not logged out since being added. Run `./setup.sh`, then log out and back in.

Port opens but no prompt

The board may still be booting. Wait a second and retry. If it persists, check for a crash loop:

```
./rftag_cli.py monitor
```

A timestamp reads 1970 or 2024

```
[1] from D1:D7:E6:13:AD:1A  group text  1970-07-29 20:45:00  (device RTC looks unset)
```

Timestamps come from the **sending** board. The RFTag has no battery-backed clock, so its RTC resets to a compiled-in default every time it loses USB power. The display flags obviously-bogus values rather than presenting them as real. This does not affect message delivery.

If a correct timestamp matters, set the clock from the host — the board has no internet or GPS time source of its own, so the host must supply it:

```
./rftag_cli.py raw rftag rtc set $(date +%s)
```

This must be repeated after every power cycle.

6. Command reference

Command	Purpose
<code>./setup.sh</code>	Install pyserial, grant serial access. Once per machine.
<code>./set_group.sh</code>	Apply group + radio profile, verify by read-back. Once per board.
<code>./set_group.sh --dry-run</code>	Show the commands without sending them.
<code>./send_message.sh <text></code>	Send one message. Repeatable.
<code>./receive_messages.sh</code>	Print everything waiting, then exit.
<code>./receive_messages.sh --watch</code>	Listen continuously until Ctrl-C.
<code>./receive_messages.sh --count</code>	Queue depth only, consumes nothing.
<code>./receive_messages.sh --clear</code>	Discard everything unread.
<code>./rftag_cli.py ports</code>	List interfaces, identify the shell.
<code>./rftag_cli.py info</code>	Firmware, MAC, group, radio config.
<code>./rftag_cli.py monitor</code>	Tail the device log console.
<code>./rftag_cli.py raw <cmd></code>	Run any firmware command verbatim.

Exit codes: **0** success, **1** failure (device rejected a command, a value failed read-back, no board found, or missing arguments).